

디지털회로및시스템설계

Ch. 11. RVX Lab. II

Instructor: Woojoo Lee

Ph.D., Associate Professor

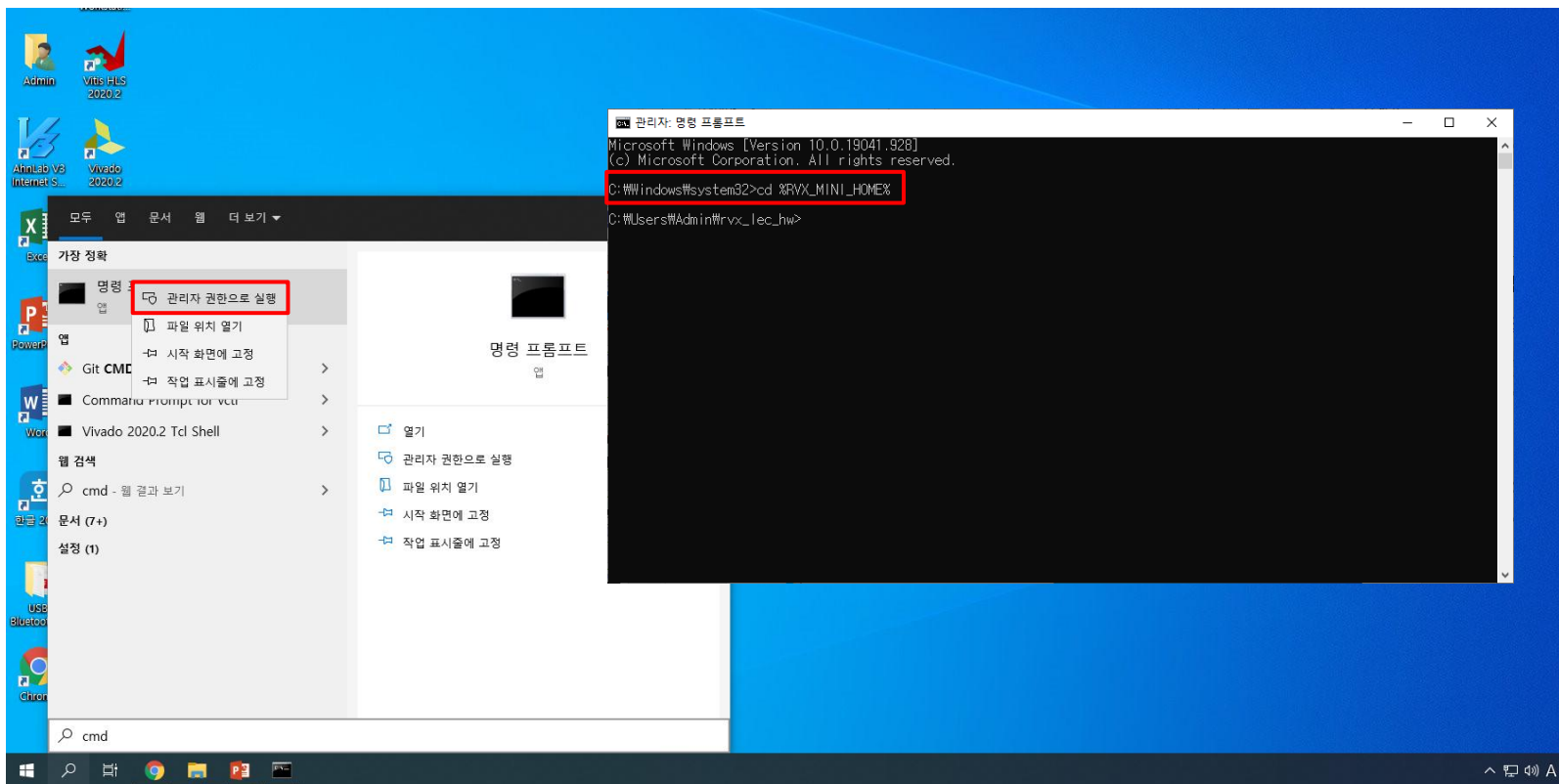
School of EEE, Chung-Ang University

RVX Application Development

명령 프롬프트 실행

1. 명령 프롬프트를 관리자 권한으로 실행
2. RVX_MINI_HOME으로 이동

```
> cd %RVX_MINI_HOME%
```



환경변수에 RVX_MINI_HOME이 정의되어 있어서 %...% 기호를 통해 사용 가능

RVX Application Development

Application 확인

```
> cd platform\Wtip_cauXX
> cd app
> dir
```

1. 이전에 생성한 platform 디렉토리로 이동
2. app 디렉토리로 이동
3. app 디렉토리 내 파일 목록 출력 및 확인

```
C:\Wrvx_lec_hw>cd platform\Wtip_cau00
C:\Wrvx_lec_hw\platform\Wtip_cau00>cd app
C:\Wrvx_lec_hw\platform\Wtip_cau00\app>dir
C 드라이브의 볼륨: OS
볼륨 일련 번호: 5AC1-643B

C:\Wrvx_lec_hw\platform\Wtip_cau00\app 디렉터리

2025-05-16 오전 08:55 <DIR> .
2025-05-21 오후 05:00 <DIR> ..
2025-05-16 오전 08:55 <DIR> bubble_sort
2025-05-16 오전 08:55 <DIR> core_check
2025-05-16 오전 08:55 <DIR> fibonacci
2025-05-16 오전 08:55 <DIR> hello
2025-05-16 오전 08:55 <DIR> 1,680 Makefile
2025-05-16 오전 08:55 <DIR> sieve
                        1개 파일                1,680 바이트
                        7개 디렉터리  573,276,680,192 바이트 남음

C:\Wrvx_lec_hw\platform\Wtip_cau00\app>
```

dir 명령어를 통해 파일 확인 가능하며, 현재 OLED 관련 application이 없는 상태

RVX Application Development

OLED application 이동

> make new APP_NAME=test_oled_bw
> dir

1. 새 application 생성
APP_NAME=test_oled_bw
2. Application DIR 생성 확인

```
C:\Wrvx lec_hw\platform\Wtip_cau00Wapp>make new APP_NAME=test_oled_bw
```

```
C:\Wrvx lec_hw\platform\Wtip_cau00Wapp>dir
```

```
C 드라이브의 볼륨: OS  
볼륨 일련 번호: 5AC1-643B
```

```
C:\Wrvx lec_hw\platform\Wtip_cau00Wapp 디렉터리
```

2025-05-21	오후 05:03	<DIR>	.
2025-05-21	오후 05:00	<DIR>	..
2025-05-16	오전 08:55	<DIR>	bubble_sort
2025-05-16	오전 08:55	<DIR>	core_check
2025-05-16	오전 08:55	<DIR>	fibonacci
2025-05-16	오전 08:55	<DIR>	hello
2025-05-16	오전 08:55		1,680 Makefile
2025-05-21	오후 05:03		270 rvx_engine.log
2025-05-16	오전 08:55	<DIR>	sieve
2025-05-21	오후 05:03	<DIR>	test_oled_bw
		2개 파일	1,950 바이트
		8개 디렉터리	573,271,015,424 바이트 남음

```
C:\Wrvx lec_hw\platform\Wtip_cau00Wapp>
```

RVX Application Development

OLED application 복사

1. lec_arty platform OLED 예제 복사하기
2. Source code 복사 확인

```
> copy ..\..\lec_arty\app\test_oled_bw\src\main.c test_oled_bw\src  
> dir test_oled_bw\src
```

```
C:\Wrvx lec_hw\platform\tip_cau00\app>copy ..\..\lec_arty\app\test_oled_bw\src\main.c test_oled_bw\src  
1개 파일이 복사되었습니다.  
  
C:\Wrvx lec_hw\platform\tip_cau00\app>dir test_oled_bw\src  
C 드라이브의 볼륨: OS  
볼륨 일련 번호: 5AC1-643B  
  
C:\Wrvx lec_hw\platform\tip_cau00\app\test_oled_bw\src 디렉터리  
  
2025-05-19 오후 04:51 <DIR> .  
2025-05-19 오후 04:48 <DIR> ..  
2025-03-13 오후 05:04 10,368 main.c  
1개 파일 10,368 바이트  
2개 디렉터리 569,220,472,832 바이트 남음  
  
C:\Wrvx lec_hw\platform\tip_cau00\app>
```

복사하는 과정은 상대위치 명령어이므로 실행되는 디렉토리가 중요
실행위치 : %RVX_MINI_HOME%\platform\tip_cau00\app

다른 디렉토리에서 실행하더라도 실행 디렉토리에 맞게 아래 형식만 만족하면 가능
copy <원본 위치와 대상> <복사본 희망 위치>

RVX Application Development

Compile 가능한 application list 출력

```
> cd ..  
> cd imp_arty-50_YYYY_MM_DD  
> make app_list
```

1. 상위 platform 디렉토리로 이동
2. FPGA imp 디렉토리로 이동
3. Compile 가능한 app list 출력

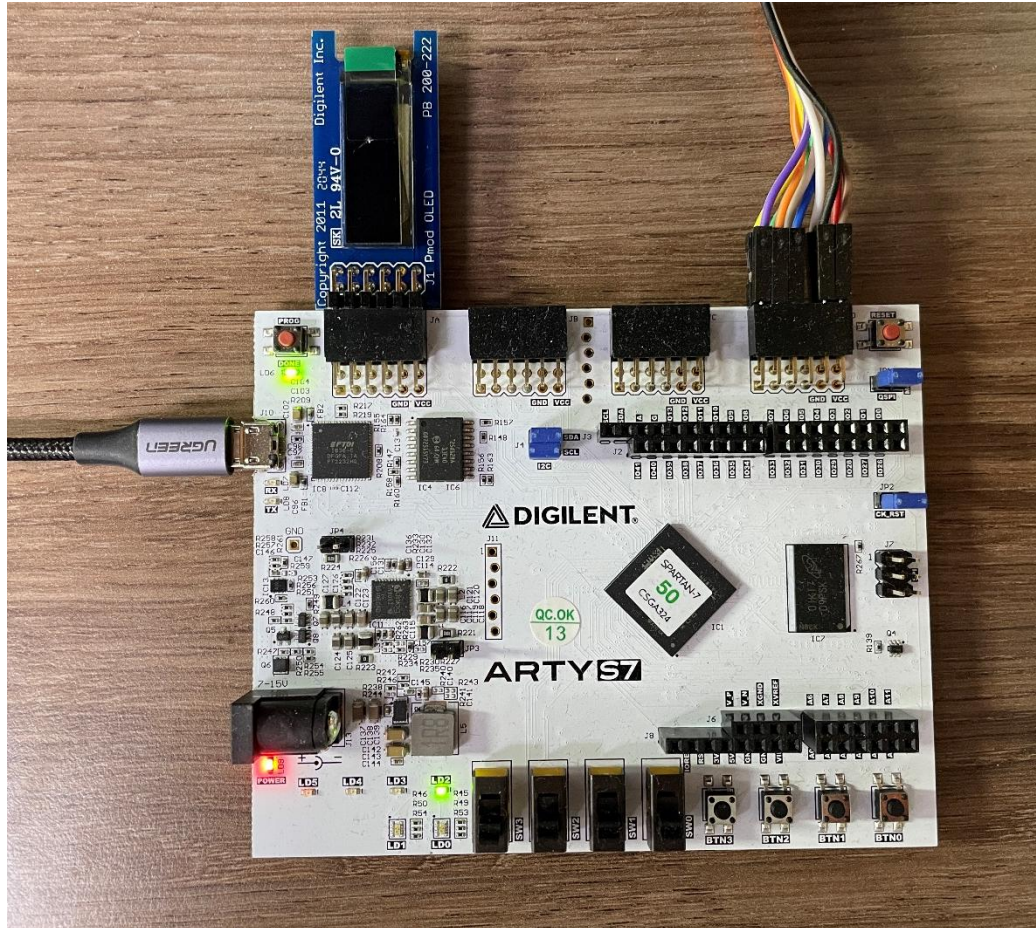
```
C:\Wrvx lec_hw\platform\tip_cau00\app>cd ..  
C:\Wrvx lec_hw\platform\tip_cau00>cd imp_arty-50_2025-05-19  
C:\Wrvx lec_hw\platform\tip_cau00\imp_arty-50_2025-05-19>make app_list  
bubble_sort  
core_check  
fibonacci  
hello  
sieve  
test_oled_bw  
C:\Wrvx lec_hw\platform\tip_cau00\imp_arty-50_2025-05-19>
```

.../tip_cau00/app 디렉토리에 있는 application들의 목록 출력
app_list에서도 새로 추가한 test_oled_bw 생성

RVX Application Development

FPGA setting

아래 사진과 같이 OLED 모듈 연결한
FPGA를 PC와 연결



RVX Application Development

FPGA program & Serial 통신, OCD 연결

> make program

bitstream 파일 program

```
C:\wrvx_lec_hw\platform\wrtip_cau00\wimp_arty-50_2025-05-19>make program

***** Vivado v2021.2 (64-bit)
**** SW Build 4029153 on Fri Oct 13 20:14:34 MDT 2023
**** IP Build 4028589 on Sat Oct 14 00:45:43 MDT 2023
**** SharedData Build 4025554 on Tue Oct 10 17:18:54 MDT 2023
** Copyright 1986-2022 Xilinx, Inc. All Rights Reserved.
** Copyright 2022-2023 Advanced Micro Devices, Inc. All Rights Reserved.
```

> make printf

> make run_oed

> make connect_oed

serial 통신, OCD 연결

```
C:\wrvx_lec_hw\platform\wrtip_cau00\wimp_arty-50_2025-05-19>make printf
C:\wrvx_lec_hw\platform\wrtip_cau00\wimp_arty-50_2025-05-19>make run_oed
C:\wrvx_lec_hw\platform\wrtip_cau00\wimp_arty-50_2025-05-19>make connect_oed
```

이전 RVX tutorial에서 실습했던 과정과 동일하게 진행

RVX Application Development

Application compile

> make test_oled_bw

1. Application compile
2. Compile 완료 확인

```
C:\rvx lec_hw\platform\tip_cau00\imp_arty-50_2025-05-19>make test_oled_bw
```

⋮

```
u00/arch/ssw/env/link.ld *.o -march=rv32im -mabi=ilp32 -static -nostartfiles -lc -lm -lgcc -Wl,-gc-sections -I/home/cau1_wjlee23/rvx_devkit/platform/tip_cau00/app/test_oled_bw/arty-50.debug/include -I/srv/rvx_shared/rvx_ssw/memorymap -I/home/cau1_wjlee23/rvx_devkit/platform/tip_cau00/arch/ssw/src -I/srv/rvx_shared/rvx_ssw/core/orca -I/srv/rvx_shared/rvx_ssw/riscv -I/srv/rvx_shared/rvx_ssw/system_utility/src -I/srv/rvx_shared/rvx_ssw/utility/src -I/srv/rvx_shared/rvx_ssw/uthash/src -I/srv/rvx_shared/rvx_ssw/image/src -I/srv/rvx_shared/rvx_ssw/matrix/src -I/srv/rvx_shared/rvx_ssw/mmio/src -I/srv/rvx_shared/rvx_ssw/api/munoc/src -I/srv/rvx_shared/rvx_ssw/api/platform_controller/src -I/srv/rvx_shared/rvx_ssw/api/uart/src -I/srv/rvx_shared/rvx_ssw/api/spi/src -I/srv/rvx_shared/rvx_ssw/api/i2c/src -I/srv/rvx_shared/rvx_ssw/api/gpio/src -I/srv/rvx_shared/rvx_ssw/api/external_peri_group/src -I/srv/rvx_shared/rvx_ssw/api/timer/src -I/srv/rvx_shared/rvx_ssw/api/oled_bw/src -I/srv/rvx_shared/rvx_ssw/api/oled_rgb/src -I/srv/rvx_shared/rvx_ssw/api/oled_bw/src -I/home/cau1_wjlee23/rvx_devkit/platform/tip_cau00/user/paco -L/home/cau1_wjlee23/rvx_devkit/platform/tip_cau00/app/test_oled_bw/arty-50.debug/etc -o /home/cau1_wjlee23/rvx_devkit/platform/tip_cau00/app/test_oled_bw/arty-50.debug/test_oled_bw.tip_cau00.elf
```

```
C:\rvx lec_hw\platform\tip_cau00\imp_arty-50_2025-05-19>
```

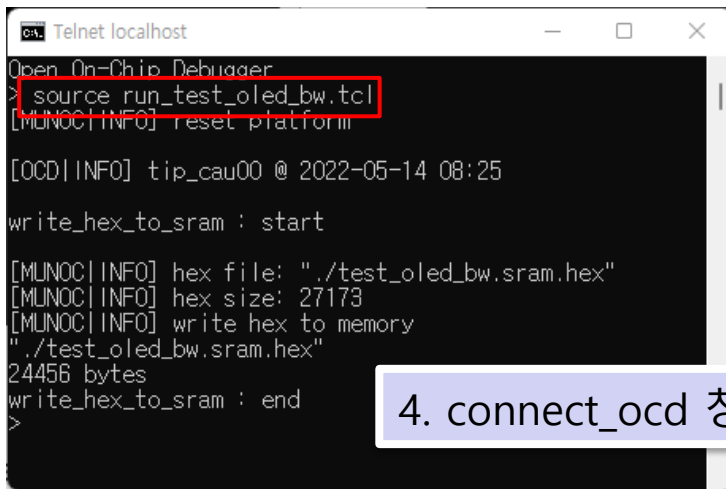
Application 정상적으로 compile 완료 시 모습

RVX Application Development

4번 connect_oed 창에서 .tcl 파일 실행

> source run_test_oled_bw.tcl

4번 connect_oed 창에서 app. 실행 command 입력



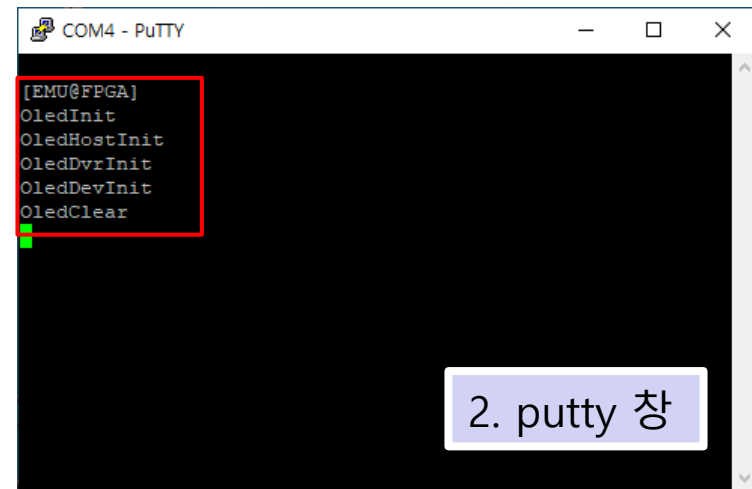
```
ca Telnet localhost
Open On-Chip Debugger
> source run_test_oled_bw.tcl
[MUNOC|INFO] reset platform

[OCD|INFO] tip_cau00 @ 2022-05-14 08:25

write_hex_to_sram : start

[MUNOC|INFO] hex file: "./test_oled_bw.sram.hex"
[MUNOC|INFO] hex size: 27173
[MUNOC|INFO] write hex to memory
"./test_oled_bw.sram.hex"
24456 bytes
write_hex_to_sram : end
>
```

4. connect_oed 창



```
COM4 - PuTTY

[EMU@FPGA]
OledInit
OledHostInit
OledDvrInit
OledDevInit
OledClear
```

2. putty 창



왼쪽 사진은 app 실행 후
OLED에 문구가 출력된 모습

RVX Application Development

Application �행 on FPGA

```
src -I/srv/rvx_shared/rvx_ssw/api/timer/src -I/srv/rvx_shared/rvx_ssw/api/oled_bw/src -I/srv/rvx_shared/rvx_ssw/api/oled_rgb/src -I/srv/rvx_shared/rvx_ssw/api/oled_bw/src -I/home/cau1_wjlee23/rvx_devkit/platform/tip_cau00/user/paco -o /srv/rvx_shared/rvx_ssw/core/orca/entry.S
/srv/rvx_shared/rvx_binary/compiler_basic/bin/riscv64-unknown-elf-gcc -T /home/cau1_wjlee23/rvx_devkit/platform/tip_cau00/arch/ssw/entry/link.ld *o -march=riscv32 -mabi=lp32 -static -nostartfiles -lc -lm -lgcc -Wl -gc-sections -I/home/cau1_wjlee23/rvx_devkit/platform/tip_cau00/app/test_oled_bw/art50-debug/include -I/srv/rvx_shared/rvx_ssw/memorymap -I/home/cau1_wjlee23/rvx_devkit/platform/tip_cau00/arch/ssw/src -I/srv/rvx_shared/rvx_ssw/core/orca -I/srv/rvx_shared/rvx_ssw/riscv -I/srv/rvx_shared/rvx_ssw/system_utility/src -I/srv/rvx_shared/rvx_ssw/utility/src -I/srv/rvx_shared/rvx_ssw/uthash/src -I/srv/rvx_shared/rvx_ssw/image/src -I/srv/rvx_shared/rvx_ssw/matrix/src -I/srv/rvx_shared/rvx_ssw/mmi/ox/src -I/srv/rvx_shared/rvx_ssw/api/munoc/src -I/srv/rvx_shared/rvx_ssw/api/platform_controller/src -I/srv/rvx_shared/rvx_ssw/api/uart/src -I/srv/rvx_shared/rvx_ssw/api/spl/src -I/srv/rvx_shared/rvx_ssw/api/i2c/src -I/srv/rvx_shared/rvx_ssw/api/gpio/src -I/srv/rvx_shared/rvx_ssw/api/external_peri_group/src -I/srv/rvx_shared/rvx_ssw/api/timer/src -I/srv/rvx_shared/rvx_ssw/api/oled_bw/src -I/srv/rvx_shared/rvx_ssw/api/oled_rgb/src -I/srv/rvx_shared/rvx_ssw/api/oled_bw/src -I/home/cau1_wjlee23/rvx_devkit/platform/tip_cau00/user/paco -L/home/cau1_wjlee23/rvx_devkit/platform/tip_cau00/art50-debug/etc -o /home/cau1_wjlee23/rvx_devkit/platform/tip_cau00/test_oled_bw.tip_cau00.elf
```

C:\Wrvx_lec_hw\platform\tip_cau00\imp_art50_2025-05-19>

1. cmd 창

```
COM4 - PuTTY

[EMU@FPGA]
OledInit
OledHostInit
OledDvrInit
OledDevInit
OledClear
```

2. putty 창

```
Open On-Chip Debugger 0.10.0-dev-snapshot (2019-04-29-16:16)
Licensed under GNU GPL v2
For bug reports, read
http://openocd.org/doc/doxygen/bugs.html
adapter speed: 3000 kHz
Info: auto-selecting first available session "jtag". To override use 'transport select <transport>'.
Info: clock speed 3000 kHz
Info: JTAG tap: rvp.noc tap/device found: 0xc47f80a1 (mfg: 0x050 (Klic), part: 0x47f8, ver: 0xc)
Info: Listening on port 3333 for gdb connections
[OCDF|INFO] tip_cau00 @ 2022-05-14 08:25
Info: Listening on port 8666 for tel connections
Info: Listening on port 4444 for telnet connections
Info: accepting 'telnet' connection on tcp/4444
[MUNOC|INFO] reset platform
[OCDF|INFO] tip_cau00 @ 2022-05-14 08:25
write_hex_to_sram : start
[MUNOC|INFO] hex file: "./test_oled_bw.sram.hex"
[MUNOC|INFO] hex size: 27173
[MUNOC|INFO] write hex to memory
"/test_oled_bw.sram.hex"
24456 bytes
write_hex_to_sram : end
```

3. run_ocd 창

```
Telnet localhost

Open On-Chip Debugger
source run_test_oled_bw.tcl
[MUNOC|INFO] reset platform

[OCDF|INFO] tip_cau00 @ 2022-05-14 08:25

write_hex_to_sram : start

[MUNOC|INFO] hex file: "./test_oled_bw.sram.hex"
[MUNOC|INFO] hex size: 27173
[MUNOC|INFO] write hex to memory
"/test_oled_bw.sram.hex"
24456 bytes
write_hex_to_sram : end
>
```

4. connect_ocd 창

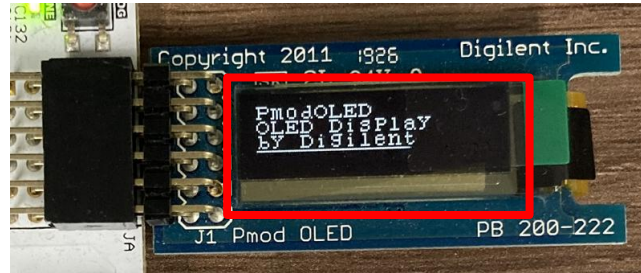
OLED application 실행 최종 모습

OLED API

OledInit()	OLED 모듈 동작을 위한 HW 세팅을 하는 함수로 한 번만 호출하면 됨.
OledSetCharUpdate(int f)	OledPutString 함수나 OledPutChar 함수를 호출할 때, 내부에서 display memory buffer에 문자를 쓴 후 자동으로 OledUpdate 함수가 실행되도록 하는 함수. 함수의 입력으로 0 아니면 1을 사용.
OledClearBuffer()	display memory buffer를 clear하는 함수
OledSetCursor(int xch, int ych)	문자를 입력할 위치를 지정하기 위해 현재cursor를 설정하는 함수. xch의 범위 0~15 ych의 범위: 0~3
OledPutString(char * sz)	현재 커서 위치부터 문자열을 display memory buffer에 쓰는 함수. 만약 문자열이 한 라인의 표현 범위를 초과하면 다음 라인에 입력됨. 함수의 수행이 완료되면 커서는 문자열 개수만큼 이동함.
OledPutChar(char ch)	현재 커서 위치부터 문자를 display memory buffer에 쓰는 함수. 함수의 수행이 완료되면 커서는 한 칸 이동함.
OledUpdate()	display memory buffer의 데이터를 OLED 모듈로 전송하여 화면에 문자열을 출력하게 해주는 함수

OLED Display 실습

.../tip_cauXX/app/test_oled_bw/src/main.c 수정



OLED 디스플레이에 본인의 이름과 학번을 띄워봅시다.

제출기한 : E-class 게시판 참조

RVX Architecture Configuration

Benchmark app 생성 및 실행

1번 cmd 창

```
> cd ../app
> make new APP_NAME=coremark
> copy ../lec_arty/app/coremark/src/* coremark/src
> cd ../imp_arty-50_YYYY-MM-DD
> make coremark
```

1. Application 새로 생성
APP_NAME=coremark
2. lec_arty platform의 coremark
source code 모두 복사
3. FPGA 디렉토리로 이동
4. coremark app. compile

4번 connect_oed 창

```
> source run_coremark.tcl
```

5. 4번 connect_oed 창에서 source

RVX Architecture Configuration

Benchmark app 생성 및 실행 명령어 화면

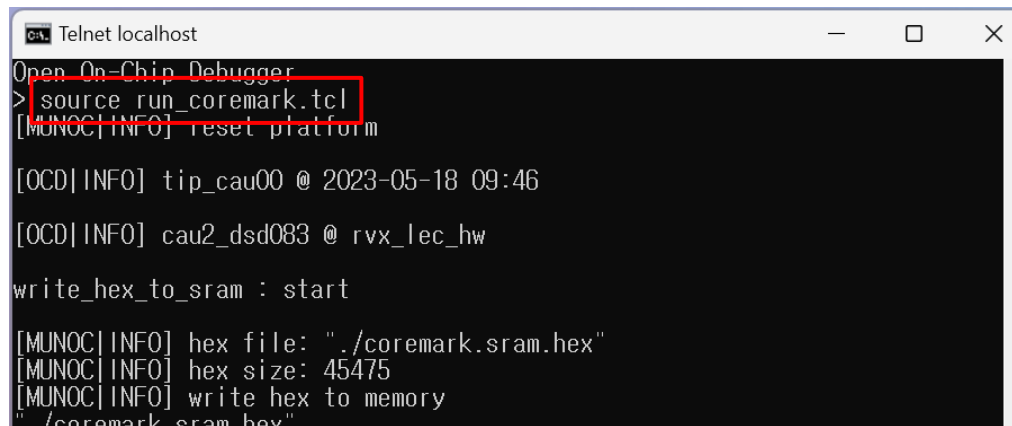
```
C:\Wrvx lec_hw\platform\Wtip_cau00\Wimp_arty-50_2025-05-19>cd ../Wapp

C:\Wrvx lec_hw\platform\Wtip_cau00\Wapp>make new APP_NAME=coremark

C:\Wrvx lec_hw\platform\Wtip_cau00\Wapp>copy ../Wlec_arty\Wapp\Wcoremark\Wsrc\W* coremark\Wsrc
../Wlec_arty\Wapp\Wcoremark\Wsrc\Wcoremark.h
../Wlec_arty\Wapp\Wcoremark\Wsrc\Wcore_list_join.c
../Wlec_arty\Wapp\Wcoremark\Wsrc\Wcore_main.c
../Wlec_arty\Wapp\Wcoremark\Wsrc\Wcore_matrix.c
../Wlec_arty\Wapp\Wcoremark\Wsrc\Wcore_portme.c
../Wlec_arty\Wapp\Wcoremark\Wsrc\Wcore_portme.h
../Wlec_arty\Wapp\Wcoremark\Wsrc\Wcore_state.c
../Wlec_arty\Wapp\Wcoremark\Wsrc\Wcore_util.c
8개 파일이 복사되었습니다.

C:\Wrvx lec_hw\platform\Wtip_cau00\Wapp>cd ../Wimp_arty-50_2025-05-19

C:\Wrvx lec_hw\platform\Wtip_cau00\Wimp_arty-50_2025-05-19>make coremark
```



```
ca Telnet localhost
Open On-Chip Debugger
> source run_coremark.tcl
[MUNOC|INFO] reset platform

[OCD|INFO] tip_cau00 @ 2023-05-18 09:46

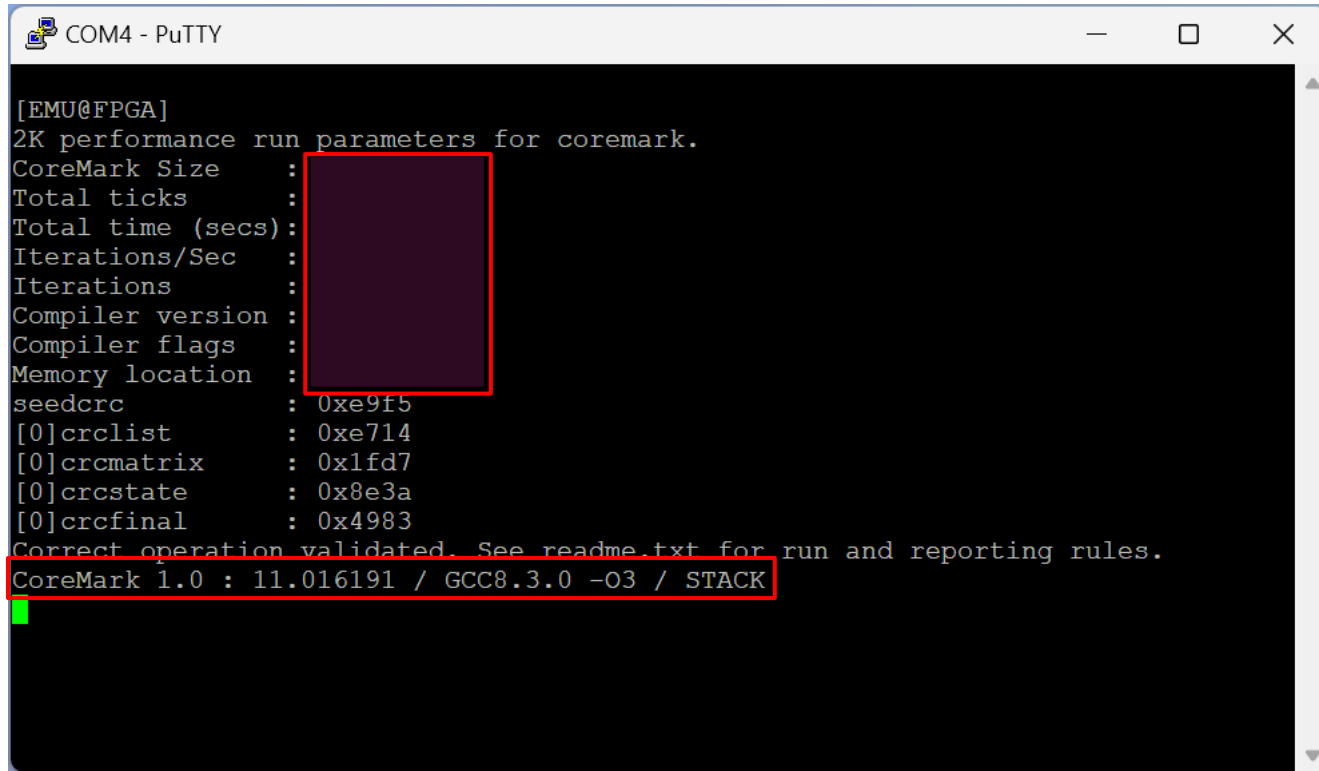
[OCD|INFO] cau2_dsd083 @ rvx lec_hw

write_hex_to_sram : start

[MUNOC|INFO] hex file: "./coremark.sram.hex"
[MUNOC|INFO] hex size: 45475
[MUNOC|INFO] write hex to memory
"/coremark.sram.hex"
```


RVX Architecture Configuration

ORCA core Benchmark 실행 결과



```
[EMU@FPGA]
2K performance run parameters for coremark.
CoreMark Size      : 
Total ticks        : 
Total time (secs)  : 
Iterations/Sec     : 
Iterations         : 
Compiler version   : 
Compiler flags     : 
Memory location    : 
seedcrc           : 0xe9f5
[0]crclist         : 0xe714
[0]crcmatrix       : 0x1fd7
[0]crcstate        : 0x8e3a
[0]crcfinal        : 0x4983
Correct operation validated. See readme.txt for run and reporting rules.
CoreMark 1.0 : 11.016191 / GCC8.3.0 -O3 / STACK
```

hello나 test_oled_bw와 달리 coremark는 출력까지 수 분 시간 소요

RVX Architecture Configuration

Core가 ORCA cache인 새 Platform 생성

```
> cd ../..
> make new PLATFORM_NAME=tip_cauXX_cache
> cd tip_cauXX_cache
> copy ../wlec_arty/wlec_arty.xml ./tip_cauXX_cache.xml
> notepad tip_cauXX_cache.xml
```

1. 새로운 platform 생성
PLATFORM_NAME= tip_cauXX_cache
2. .xml 복사 및 수정

앞서 실습한 Coremark application을 활용하여 cache 유무의 차이점을 확인하기 위해
ORCA cache를 core로 하는 새로운 platform 생성

RVX Architecture Configuration

Core가 ORCA cache인 새 Platform 생성

The image shows an XML editor window with two tabs: 'lec_cau00_cache.xml' and 'tip_cau00.xml'. The 'tip_cau00.xml' tab is active, displaying the following XML content:

```
<?xml version="1.0" ?>
<rvx>
  <platform>
    <name>tip_cau00_cache</name>
    <status>described</status>
    <spec>
      <define>
        <name>sram_size</name>
        <value>128kB</value>
      </define>
      ...
      <define>
        <name>num_led_checker</name>
        <value>1</value>
      </define>
    </spec>
    <ip_instance>
      <name>i_main_core</name>
      <library_name>rvc_orca_cache</library_name>
    </ip_instance>
  </platform>
</rvx>
```

Annotations in the image indicate the following changes:

- platform 이름과 동일하게 수정** (Change platform name to be the same): This annotation points to the `<name>tip_cau00_cache</name>` line, which is highlighted with a red box.
- Core 이름 rvc_orca_cache로 수정** (Change Core name to rvc_orca_cache): This annotation points to the `<library_name>rvc_orca_cache</library_name>` line, which is highlighted with a red box.

The status bar at the bottom of the editor shows: 줄 46, 열 32 | 1,025자 | 100% | Windows (CRLF) | UTF-8.

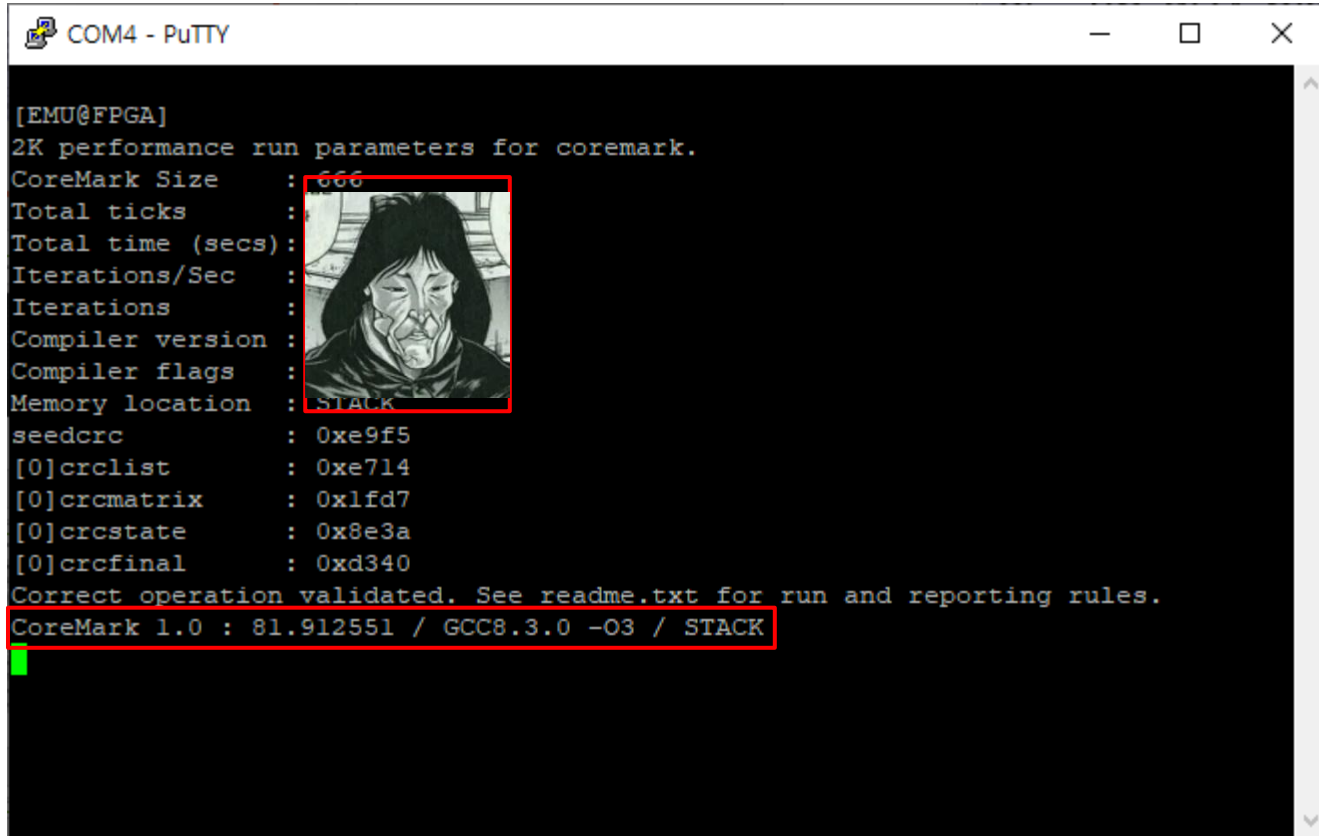
RVX Architecture Configuration

Benchmark app 생성 및 실행

1. platform 합성
2. 기본 testbench 생성
3. FPGA 디렉토리 생성
4. bitstream 생성
5. FPGA program
6. FPGA Serial 통신 & OCD 연결
7. 새 Application 생성 APP_NAME=coremark
8. lec_arty platform의 coremark source code 모두 복사
9. FPGA 디렉토리로 이동하여 coremark application 컴파일
10. 4번 connect_oed 창에서 source

각 단계에 필요한 명령어는 이전 강의 내용 참조

ORCA cache core Benchmark 실행 결과 & 비교



```
[EMU@FPGA]
2K performance run parameters for coremark.
CoreMark Size      : 666
Total ticks        :
Total time (secs)  :
Iterations/Sec     :
Iterations         :
Compiler version   :
Compiler flags     :
Memory location    : STACK
seedcrc           : 0xe9f5
[0]crclist        : 0xe714
[0]crcmatrix      : 0x1fd7
[0]crcstate       : 0x8e3a
[0]crcfinal       : 0xd340
Correct operation validated. See readme.txt for run and reporting rules.
CoreMark 1.0 : 81.912551 / GCC8.3.0 -O3 / STACK
```

Cache가 없는 이전 platform과 cache가 있는 현재 platform 결과 비교

제출기한: E-class 게시판 참조

RVX Tip.

RVX Update

```
> cd %RVX_MINI_HOME%  
> update.bat
```

1. rvx_lec_hw 폴더 backup
2. RVX home으로 이동
3. update.bat 실행

```
C:\Wrvx_lec_hw>update.bat  
  
C:\Wrvx_lec_hw>git pull origin master  
Unpacking objects: 100% (14/14), 3.33 KiB | 262.00 KiB/s, done.  
From https://bitbucket.org/kyuseung_han/rvx_lec_hw  
* branch          master      -> FETCH_HEAD  
  9f3e11c..111b93e  master      -> origin/master  
Fetching submodule rvx_init  
From https://bitbucket.org/kyuseung_han/rvx_init  
  d59ac3c..f7a39a7  master      -> origin/master  
Fetching submodule rvx_install  
From https://bitbucket.org/kyuseung_han/rvx_install  
  027703f..0c7bcfc  master      -> origin/master  
Updating 9f3e11c..111b93e  
Fast-forward  
 platform/tip_hello/Makefile          | 96 ++++++  
 platform/tip_hello/app/.gitattributes | 1 +  
 platform/tip_hello/app/Makefile       | 43 ++++++  
 platform/tip_hello/app/hello/Makefile | 52 ++++++  
 platform/tip_hello/app/hello/compile_list | 3 +  
 platform/tip_hello/app/hello/src/main.c | 8 +++  
 platform/tip_hello/tip_hello.xml      | 19 +++++  
 rvx_init                             | 2 +-  
 rvx_install                           | 2 +-  
 9 files changed, 224 insertions(+), 2 deletions(-)
```

update.bat 파일이 없을 경우 make config 명령어를 통해 생성
update를 시작하면 초기 설정 명령어가 자동으로 연속 실행

RVX Tip.

RVX Reset

```
> cd %RVX_MINI_HOME%  
> make clean
```

1. rvx_lec_hw 폴더 backup
2. RVX home으로 이동
3. make clean 입력

```
C:\rvx_lec_hw>make clean
```

```
C:\rvx_lec_hw>
```

Reset 후에는 RVX install 첫 과정부터 다시 시작